

Capstone Project Report

Selenium Python Framework Development using PyTest, Unittest and Page Object Model

Application Under Test: TutorialsNinja Demo

1. Project Overview

This capstone project implements a maintainable web automation testing framework using Python and Selenium WebDriver. The framework integrates PyTest and Python's built-in Unittest framework and follows the Page Object Model (POM) design pattern. It automates registration, login, product search, add to cart, cart validation, and logout workflows.

2. Problem Statement

Manual execution of repetitive web application test scenarios requires significant time and may result in inconsistent execution and reporting. The project addresses this by providing repeatable automated execution, reusable page objects, centralized setup and cleanup, failure screenshots, and HTML reporting.

3. Objectives

Develop a Python Selenium framework; apply Page Object Model; use PyTest fixtures and assertions; demonstrate Unittest integration; use explicit waits; separate configuration and test data; capture failure screenshots; generate an HTML report; and automate core TutorialsNinja workflows.

4. Tools, Software and Technologies

Python 3.13.5, Selenium WebDriver, PyTest, Python Unittest, pytest-html, Google Chrome, Visual Studio Code, and the TutorialsNinja Demo application.

5. Framework Architecture

The framework is organized into configuration, test data, page objects, tests, utilities, reporting, screenshots, and documentation. `config/config.py` stores the application URL. `data/test_data.py` stores reusable credentials and product data. The `pages` directory contains `HomePage`, `LoginPage`, `RegisterPage`, `ProductPage`, and `AccountPage`. `conftest.py` provides the browser fixture and failure screenshot handling. `pytest.ini` configures discovery and HTML reporting.

6. Page Object Model Implementation

Each major application page is represented by a Python class containing its locators and reusable browser actions. This separates UI interaction from test logic, reduces duplication, and improves maintainability.

7. Automated Test Scenarios

TC01 User Registration — PyTest — PASS. TC02 Valid User Login — PyTest — PASS. TC03 Product Search — PyTest — PASS. TC04 Add Product to Cart — PyTest — PASS. TC05 Cart Validation — PyTest — PASS. TC06 User Logout — PyTest — PASS. TC07 Product Search — Unittest — PASS.

8. Synchronization and Reliability

Explicit waits using `WebDriverWait` and Selenium expected conditions are used to synchronize browser actions with the application. This avoids unnecessary fixed delays and waits for elements to become visible or clickable before interaction.

9. Test Data and Configuration Management

Application configuration and reusable test values are maintained outside the main test logic. The base URL is stored in `config/config.py`, while reusable login credentials and product information are stored in `data/test_data.py`.

10. Failure Screenshot Handling

The PyTest fixture checks the execution result after each test. When a test fails, a screenshot is automatically saved in the capstone Screenshots directory, providing visual evidence for debugging and analysis.

11. HTML Reporting

`pytest.ini` enables `pytest-html` and generates a self-contained HTML execution report in the Reports directory. The report records the tests executed and their results.

12. Execution Result

The complete PyTest suite was executed successfully with 7 passed tests and 0 failed tests in approximately 57.65 seconds. The execution also generated the configured HTML report.

13. Result

The project successfully delivered a functional Selenium Python automation framework using PyTest and Unittest, reusable page objects, browser fixtures, explicit waits, centralized configuration and test data, failure screenshots, and HTML reporting.

14. Observation

The Page Object Model reduced duplication and improved maintainability. Centralized test data simplified updates, explicit waits improved synchronization, PyTest simplified execution and reporting, and Unittest demonstrated compatibility with Python's built-in testing framework.

15. Conclusion

The project demonstrates the practical use of Selenium WebDriver with Python for structured web automation. The framework provides a solid foundation for additional scenarios, data-driven execution, cross-browser testing, and future CI integration.

16. Project Deliverables

Complete Selenium Python source code; Page Object Model classes; PyTest test suite; Unittest demonstration; centralized configuration and test data; failure screenshot mechanism; PyTest HTML report; test case documentation; screenshots; and demonstration materials.

17. Final Execution Summary

Metric	Result
PyTest tests	6 passed
Unittest tests	1 passed
Total automated tests	7
Failed tests	0
Pass percentage	100%